

编译原理第八章—代码优化

考试复习总结

中南大学计算机学院

2026 年 5 月

内容摘要：代码优化的层次与分类、基本块与流图、
七种常见优化技术、循环优化、数据流分析基础（到达定值 / 活跃变量 / 可用表达式）

目录

1	代码优化概述	2
2	基本块与流图	3
3	常见的局部优化技术	4
4	循环优化	8
5	全局优化与数据流分析基础	11
6	核心考点速记	14

1 代码优化概述

8.1.1 什么是代码优化

代码优化（**Code Optimization**）是指在保持程序语义不变的前提下，对程序进行各种等价变换，使得变换后的程序能够生成更有效的目标代码 [p.3]。

优化不是编译过程的必需阶段，但对生成高质量目标代码至关重要。优化可以在**中间代码**上进行（与机器无关），也可以在**目标代码**上进行（依赖具体机器）。

8.1.2 代码优化的三项原则

代码优化必须遵循以下三条基本原则 [p.3]：

- (1) **等价原则（Equivalence Principle）**：经过优化后的程序不应改变程序原有的运行结果。这是优化的根本前提——任何优化都不能引入语义错误。
- (2) **有效原则（Effectiveness Principle）**：优化后所产生的目标代码运行时间较短，占用的存储空间较小。时间优化和空间优化往往是优化的两个核心指标，两者有时需要权衡。
- (3) **合算原则（Cost-Effectiveness Principle）**：应尽可能以较低的代价取得较好的优化结果。如果一项优化的开销（编译时间、算法复杂度）远超其带来的运行时收益，则该优化在实际编译器中不值得实施。

8.1.3 优化的分类

优化可以从**阶段**和**范围**两个维度进行划分 [p.4]：

表 1: 代码优化的分类

分类维度	类别	说明
按阶段划分	源代码级优化（Source-level）	在源代码层次进行，如内联展开
	中间代码级优化（IR-level）	在中间表示上进行，与机器无关
	目标代码级优化（Target-level）	依赖目标机器特性，如寄存器分配、指令调度
按范围划分	局部优化（Local）	在 基本块 内进行的优化 [p.4]
	循环优化（Loop）	在 循环体 内进行的优化 [p.4]
	全局优化（Global）	在整个 程序范围 内进行的优化 [p.4]

补充说明：

- 局部优化是最基本的优化，处理范围小但频率高，开销较低。
- 循环优化尤为重要，因为程序的大量执行时间通常集中在循环中（著名的 90/10 法则）。
- 全局优化需要跨基本块的数据流分析信息，编译开销最大，但收益也可能最高。

2 基本块与流图

8.2.1 基本块 (Basic Block)

基本块是指程序中一个顺序执行的语句序列，其中只有一个入口语句（第一个语句）和一个出口语句（最后一个语句）。基本块是局部优化的基本单元。

基本块的划分规则：

1. 确定入口语句 (Leader)：

- 程序的第一条语句是入口语句。
- 任何能由条件转移语句或无条件转移语句转到的语句是入口语句。
- 紧跟在转移语句后面的语句是入口语句。

2. 构造基本块：每个入口语句及其到下一个入口语句之前（或程序结束）的所有语句组成一个基本块。

8.2.2 流图 (Flow Graph)

流图是一个有向图 $G = (N, E, Entry, Exit)$ ，其中：

- N ：节点集合，每个节点代表一个**基本块**。
- E ：边集合，表示基本块之间的控制流转移关系。
- $Entry$ ：唯一的入口节点（程序起点）。
- $Exit$ ：唯一的出口节点（程序终点）。

流图中边的添加规则：

- 若基本块 B_1 的出口语句是跳转到 B_2 入口的转移语句，则从 B_1 到 B_2 加一条有向边。
- 若 B_1 的出口不是无条件转移且 B_2 紧跟在 B_1 之后，则从 B_1 到 B_2 加一条有向边（表示顺序执行）。

流图是进行**全局数据流分析**的基础数据结构。通过流图可以清晰地识别出循环结构、支配关系等，为后续优化提供依据。

3 常见的局部优化技术

以下优化技术以中间代码（三地址码 / 四元式）为操作对象，是最基本、最重要的优化手段。

8.3.1 删除公共子表达式（Common Subexpression Elimination, CSE）

定义：如果一个表达式 E 在前面已经计算过，并且在这之后 E 中各变量的值没有改变，则称 E 为公共子表达式（Common Subexpression）。对于公共子表达式，可以避免其重复计算，直接引用前面计算的结果，称为删除公共子表达式，也称删除多余运算 [p.5]。

作用：减少重复计算，节省运行时间。

示例 [p.6]：

优化前：

```
(1) T6 := 4*i
(2) X  := a[T6]
(3) T7 := 4*i          // 4*i 前面已计算，是公共子表达式
(4) T8 := 4*j
(5) T9 := a[T8]
(6) a[T7] := T9
(8) T10 := 4*j         // 4*j 前面已计算，是公共子表达式
(9) a[T10] := x
(10) goto B2
```

优化后（第一轮）：

```
(1) T6 := 4*i
(3) T7 := T6          // 用 T6 替换 4*i
(4) T8 := 4*j
(8) T10 := T8         // 用 T8 替换 4*j
```

优化后（进一步，结合复写传播）：

```
(1) T2 := 4*i        // 用临时变量保存公共值
(1) T4 := 4*j
(1) T6 := T2
(3) T7 := T6
(4) T8 := T4
(8) T10 := T8
```

考试注意：公共子表达式可以是涉及多个变量的复杂表达式（如 $a+b*c$ ），判断“公共”的条件是：表达式结构相同且操作数在两次出现之间未被修改。

8.3.2 复写传播（Copy Propagation）

定义：当程序中出现了形如 $f := g$ 的复写语句（copy statement）后，在后续代码中尽可能用 g 代替 f 的使用，这种变换称为复写传播 [p.7]。

作用：

- 消除冗余的中间变量引用，使代码更直接。
- 为后续的死代码消除创造条件（复写语句本身可能变成无用代码）。

示例 [p.7]:

优化前:

```
(1) T6 := T2
(2) X  := a[T6]
(3) T7 := T6
(4) T8 := T4
(5) T9 := a[T8]
(6) a[T7] := T9
(8) T10 := T8
(9) a[T10] := x
(10) goto B2
```

优化后（第一轮传播）:

```
(2) X  := a[T2]          // 用 T2 替换 T6
(3) T7 := T2
(5) T9 := a[T4]          // 用 T4 替换 T8
(6) a[T2] := T9          // 用 T2 替换 T7
(8) T10 := T4
(9) a[T4] := x
```

优化后（第二轮传播，引入 T3, T5 已知值）:

```
(2) X  := T3             // a[T2] 的值已知为 T3
(5) T9 := T5             // a[T4] 的值已知为 T5
(6) a[T2] := T5
(9) a[T4] := T3
```

关键点: 复写传播后，原来的复写语句（如 $T6 := T2$ ）变成对 $T6$ 的赋值为无用，因为后续代码不再引用 $T6$ ，可通过死代码删除消除它们。

8.3.3 删除死代码（Dead Code Elimination / 删除无用代码）

定义: 有些变量在赋值后从未被引用，这些变量的赋值对程序运算结果没有任何作用，可以删除对这些变量赋值的代码，称为删除无用代码或删除死代码（Dead Code Elimination）[p.8]。

两类死代码:

1. **无用赋值（Dead Assignment）:** 变量被赋值后再也不被使用。通常在复写传播后大量产生。
2. **不可达代码（Unreachable Code）:** 控制流永远无法到达的代码块。例如紧跟在无条件跳转之后且无标号的语句。

示例 [p.8]:

优化前（复写传播后）:

```

(1) T6 := T2           // T6 在后续中不再被引用 → 死代码
(2) X  := T3
(3) T7 := T2           // T7 在后续中不再被引用 → 死代码
(4) T8 := T4           // T8 在后续中不再被引用 → 死代码
(5) T9 := T5           // T9 在后续中不再被引用 → 死代码
(6) a[T2] := T5
(8) T10 := T4          // T10 在后续中不再被引用 → 死代码
(9) a[T4] := T3
(10) goto B2

```

优化后：

```

(6) a[T2] := T5
(8) a[T4] := T3
(10) goto B2

```

综合效果：通过 CSE → 复写传播 → 死代码消除这条流水线，可将原 10 条指令缩减为 3 条，体现了三种优化联合使用时的叠加效应。

8.3.4 合并已知量 (Constant Folding)

定义：对于那些在编译时便可静态确定的运算结果，在编译时就计算出来，不必等到运行程序时再计算，这种优化变换称为合并已知量 [p.12]。也常称为常量折叠。

示例 [p.12]：

优化前：

```

T1 := 2
.....
T4 := 4 * T1

```

优化后：

```

T1 := 2
.....
T4 := 8           // 编译时计算 4*2=8

```

扩展应用：

- 常量表达式计算：a := 3 + 5 → a := 8。
- 布尔常量折叠：if (true and false) → if (false)。
- 条件常量传播：结合常量传播，在条件分支中若条件为常量则可消除不可达分支。

8.3.5 代数化简 (Algebraic Simplification)

定义：对一些求值的表达式，用代数上等价的形式替换，使复杂运算变成简单运算，称为代数变换或代数化简 [p.13]。

常见代数恒等式：

- 恒等运算: $x + 0 = x$, $x - 0 = x$, $x \times 1 = x$, $x \div 1 = x$
- 零元运算: $x \times 0 = 0$, $0 \div x = 0$ ($x \neq 0$)
- 冗余运算: $x \times 2 = x + x$, $x^2 = x \times x$ (用乘法代替幂运算)
- 强度削弱相关: $2 \times x = x + x$, $x \div 2 = x \times 0.5$

示例 [p.13]:

优化前:

```
X := X + 0           // 恒等运算, 可删除
X := X * 1           // 恒等运算, 可删除
.....
X := Y ** 2          // 幂运算, 代价高
```

优化后:

```
X := Y * Y           // 用乘法代替幂运算
```

注意: 代数化简需要小心浮点数精度问题。有些代数等价式在浮点运算下可能不严格等价 (如 $a + (b + c) \neq (a + b) + c$ 在浮点下)。

8.3.6 常量传播 (Constant Propagation)

定义: 如果在编译时能确定某变量在程序某点的值是一个常量, 则可将该变量在该点的使用直接替换为该常量。这实际上是复写传播的特例 (源操作数为常量)。

示例:

```
pi := 3.14159
circumference := 2 * pi * r
```

常量传播后编译器可在编译时完全计算出以常量为参数的表达式。

8.3.7 局部优化的组织: DAG 表示

在基本块内实施局部优化时, 通常采用有向无环图 (DAG, Directed Acyclic Graph) 来表示基本块内的计算依赖关系。

DAG 的构造规则:

- 叶节点: 以变量名或常量标记 (若为常量则标值)。
- 内部节点: 以运算符标记, 代表运算结果。
- 附加标识符: 每个节点可附加多个变量名, 表示这些变量当前都持有该节点的值。

通过 DAG 可实现:

- 自动检测公共子表达式 (同一表达式出现时对应同一节点)。
- 确定哪些变量在基本块内被引用 (活跃信息)。
- 确定哪些计算可在基本块外使用 (输出节点)。

4 循环优化

循环是程序中最耗时的部分。据统计，程序中约 90% 的执行时间消耗在 10% 的代码上，而这 10% 往往正是循环。因此循环优化对整体性能提升至关重要。

循环的识别：在流图中，循环对应一组节点构成的**强连通分量**（SCC），具有以下性质：

- 有唯一的入口节点（循环头 / Loop Header）。
- 循环中每个节点都有到达循环头的路径。
- 从循环外进入循环必须经过循环头。

8.4.1 代码外提（Code Hoisting / Loop-Invariant Code Motion）

定义：将循环体中**不随循环变化的运算**（循环不变量，Loop Invariant）提到循环体的前面（循环之前），使之只在进入循环时计算一次，而不是每次迭代都重复计算 [p.9]。

循环不变量的判定：运算 $A := B \text{ op } C$ 是循环不变量，当且仅当：

1. B 和 C 都是常数；或
2. 到达 A 的 B 和 C 的定值点都在循环之外；或
3. 虽然 B 或 C 在循环内被定值，但该定值本身也是循环不变量。

示例 [p.9]：

优化前：

```
while (i <= limit - 2) {      // limit-2 每次迭代都重新计算
    .....
    i := i + 1
}
```

优化后：

```
t := limit - 2                // 提到循环外，只计算一次
while (i <= t) {
    .....
    i := i + 1
}
```

代码外提的条件（安全性）：

- 外提的语句必须支配循环的所有出口（即循环每次都会执行该语句）。
- 若循环可能不执行（如 while 条件初始为假），外提的语句不能影响程序语义。
- 该变量在循环外不被引用时才能安全外提（否则即使循环不执行，变量也应保持原值）。

8.4.2 强度削弱 (Strength Reduction)

定义：加减法运算一般比乘除法快。把循环中的乘法等**高强度运算**替换为加法等**低强度运算**，以提高运算效率，称为**强度削弱** [p.10]。

核心思想：对于循环中的**归纳变量** (Induction Variable, 每次迭代按固定步长变化的变量)，将其相关运算转换为递推形式的加法运算。

示例 [p.10]：

优化前：

```
j := j - 1
T4 := 4 * j           // 乘法：高强度运算
T5 := a[T4]
if T5 > v goto B3
```

优化后：

```
j := j - 1
T4 := T4 - 4          // 加减法：低强度运算（用递推代替乘法）
T5 := a[T4]
if T5 > v goto B3
```

原理：若 j 每次减少 1，则 $T4 = 4 \times j$ 每次减少 4，因此 $T4_{\text{new}} = T4_{\text{old}} - 4$ ，避免了每次迭代重新计算 $4 \times j$ 。

强度削弱的常见模式：

- 乘法转加法： $i \times c$ 可替换为每次加 c (c 为常量)。
- 除法转乘法： $i \div c \rightarrow i \times (1/c)$ (浮点)。
- 乘方转乘法： $x^2 \rightarrow x \times x$ 。
- 下标地址计算：将 $addr = base + i \times stride$ 改为递推 $addr = addr + stride$ 。

8.4.3 归纳变量消除 (Induction Variable Elimination)

定义：通过变换循环控制条件，删除那些纯粹为控制循环而设立的归纳变量，达到优化目的 [p.11]。

基本归纳变量 vs 派生归纳变量：

- **基本归纳变量 (Basic Induction Variable)：**其值在循环中仅由形如 $i := i \pm c$ 的语句改变 (c 为循环不变量)。例如循环计数器 i 。
- **派生归纳变量 (Derived Induction Variable)：**其值是基本归纳变量的线性函数，形如 $j := k \times i + b$ 。例如 $T2 := 4 \times i$ 。

优化策略：

1. 若基本归纳变量仅用于控制循环，且存在同族的派生归纳变量，可用派生归纳变量替换基本归纳变量作为循环控制条件。
2. 若基本归纳变量还用于其他目的，可保留但不能消除。

示例 [p.11]:

优化前:

```
i := m - 1
j := n
T2 := 4 * i
T4 := 4 * j
L:  if (i >= j) goto done    // 用 i, j 控制循环
    .....
    i := i - 1              // 基本归纳变量
    j := j - 1
    T2 := T2 - 4
    T4 := T4 - 4
    goto L
```

优化后:

```
T2 := 4 * i
T4 := 4 * j
L:  if (T2 >= T4) goto done  // 用派生归纳变量 T2, T4 控制循环
    .....
    T2 := T2 - 4
    T4 := T4 - 4
    goto L
```

注意: 原本 $i := i - 1$ 和 $j := j - 1$ 在优化后可能变成死代码 (若 i 、 j 后续不再被引用), 可以进一步消除。

5 全局优化与数据流分析基础

8.5.1 全局优化概述

全局优化 (Global Optimization) 是在整个程序范围 (跨基本块) 内进行的优化。与局部优化不同, 全局优化需要考虑不同基本块之间的控制流和数据依赖关系。典型的全局优化包括:

- 全局公共子表达式消除
- 全局复写传播
- 全局死代码消除
- 循环不变量的全局检测与外提

全局优化的基础是**数据流分析 (Data Flow Analysis)**, 通过分析程序中数据的定义、使用和传播情况, 获得优化所需的信息。

8.5.2 基本概念

- **定值 (Definition, def)**: 对变量赋值 (包括赋值语句、输入语句等), 使变量的值被定义。
- **引用 (Use)**: 在表达式中读取变量的值。
- **到达 (Reach)**: 一个定值 d 到达某点 p , 当且仅当存在一条从 d 到 p 的路径, 且在这条路径上该变量没有被重新定值 (没有被“杀死”)。
- **活跃 (Live)**: 变量在某点活跃, 当且仅当存在一条从该点出发的路径, 在该路径上该变量的当前值被引用 (在被重新定值之前)。
- **可用 (Available)**: 表达式在某点可用, 当且仅当每条到达该点的路径上该表达式都被计算过, 且在计算后其操作数未被修改。

8.5.3 到达定值分析 (Reaching Definitions)

目的: 确定在程序的每个点上, 哪些变量的定值可能到达该点。这是最基础的数据流分析, 用于常量传播、复写传播等。

数据流方程 (前向分析):

$$\begin{aligned} \text{OUT}[B] &= \text{GEN}[B] \cup (\text{IN}[B] - \text{KILL}[B]) \\ \text{IN}[B] &= \bigcup_{P \in \text{pred}(B)} \text{OUT}[P] \end{aligned}$$

其中:

- $\text{GEN}[B]$: 基本块 B 中定义的且在 B 出口仍有效的定值集合。
- $\text{KILL}[B]$: 基本块 B 中因新定值而被“杀死” (覆盖) 的所有其他定值。
- $\text{IN}[B]$: 到达 B 入口的定值集合。

- $OUT[B]$: 从 B 出口流出的定值集合。

初始条件: $OUT[Entry] = \emptyset$, 其余 $OUT[B] = \emptyset$ 。

迭代求解: 反复计算每个基本块的 IN 和 OUT, 直到所有 OUT 不再变化 (到达不动点)。

应用:

- **常量传播**: 若某点只有唯一定值到达, 且该定值为常量, 则可替换为常量。
- **复写传播**: 若某点只有唯一定值 $x := y$ 到达, 则可用 y 替换 x 。

8.5.4 活跃变量分析 (Live Variable Analysis)

目的: 确定在程序每个点上, 哪些变量的值在后续执行中可能被引用。用于寄存器分配和死代码消除。

数据流方程 (后向分析):

$$\begin{aligned} IN[B] &= USE[B] \cup (OUT[B] - DEF[B]) \\ OUT[B] &= \bigcup_{S \in succ(B)} IN[S] \end{aligned}$$

其中:

- $USE[B]$: 基本块 B 中在被定值之前被引用的变量集合。
- $DEF[B]$: 基本块 B 中在被引用之前被定值的变量集合。
- $IN[B]$: B 入口处活跃的变量集合。
- $OUT[B]$: B 出口处活跃的变量集合。

初始条件: $IN[Exit] = \emptyset$, 其余 $IN[B] = \emptyset$ 。

应用:

- **寄存器分配**: 两个变量若活跃区间不重叠, 可分配到同一寄存器。
- **死代码消除**: 若变量在赋值后不再活跃 (出口不活跃), 则该赋值是死代码。

8.5.5 可用表达式分析 (Available Expressions)

目的: 确定在程序每个点上, 哪些表达式的值已经被计算且操作数未被修改, 从而可以直接引用。用于全局公共子表达式消除。

数据流方程 (前向分析):

$$\begin{aligned} OUT[B] &= EGEN[B] \cup (IN[B] - EKILL[B]) \\ IN[B] &= \bigcap_{P \in pred(B)} OUT[P] \quad (\text{注意: 交集, 非并集!}) \end{aligned}$$

其中:

- $EGEN[B]$: B 中计算的且之后操作数未被修改的表达式。
- $EKILL[B]$: B 中因操作数被修改而失效的表达式。

与到达定值的区别: 可用表达式分析中 IN 是交集 (每条路径上都可用才可用), 而非并集。因为要保证”一定被计算过”, 而非”可能被计算过”。

应用: 若表达式 E 在某点可用, 则可用之前计算的值得替换 E , 消除重复计算。

8.5.6 三类数据流分析对比

表 2: 三种主要数据流分析的对比

分析类型	方向	交汇运算	主要用途
到达定值	前向	$\cup$ (并集)	常量传播、复写传播
活跃变量	后向	$\cup$ (并集)	寄存器分配、死代码消除
可用表达式	前向	$\cap$ (交集)	全局公共子表达式消除

8.5.7 UD 链与 DU 链

UD 链 (Use-Definition Chain / 引用-定值链): 对于变量的每次引用, UD 链给出所有可能到达该引用的定值。回答”这个值可能从哪里来?”。

DU 链 (Definition-Use Chain / 定值-引用链): 对于变量的每个定值, DU 链给出所有可能被该定值影响的引用。回答”这个定值可能用到哪里?”。

UD/DU 链由到达定值分析的结果构建, 是许多优化 (如死代码消除、循环不变外提) 的直接输入。

6 核心考点速记

考点 1：代码优化的三项原则

等价原则	优化不能改变程序的运行结果 [p.3]
有效原则	目标代码更快、占用空间更小 [p.3]
合算原则	以较低代价取得较好结果 [p.3]

考点 2：优化分类

维度	类别	示例
阶段	源代码级 → 中间代码级 → 目标代码级 [p.4]	内联展开 → CSE → 寄存器分配
范围	局部 → 循环 → 全局 [p.4]	基本块内 → 循环体 → 跨基本块

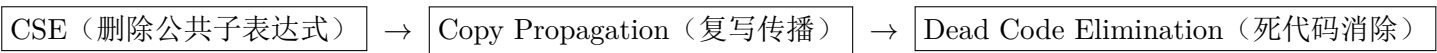
考点 3：七种经典优化技术一览

表 3: 七种经典优化技术汇总

优化名称	定义/条件	示例
删除公共子表达式 (CSE) [p.5-6]	表达式前面算过且操作数未变	T7:=4*i → T7:=T6
复写传播 [p.7]	f:=g 后, 用 g 替代 f	a[T6] → a[T2]
删除死代码 [p.8]	变量赋值后不再被引用	删除无用的 T6:=T2 等
代码外提 [p.9]	循环不变运算提到循环外	t:=limit-2; 放到 while 前
强度削弱 [p.10]	乘法等转为加法等	T4:=4*j → T4:=T4-4
归纳变量消除 [p.11]	用派生归纳变量替换基本归纳变量	if(i>=j) → if(T2>=T4)
合并已知量 [p.12]	编译时计算常量表达式	T4:=4*2 → T4:=8
代数化简 [p.13]	用等价代数式替换	X:=Y**2 → X:=Y*Y

考点 4：优化流水线

常见的优化执行顺序（课件给出了一条流水线）：



这三步联合使用可产生显著的代码缩减效果 [p.5-8]。

考点 5：循环优化三步曲

1. 代码外提：将循环不变量移到循环之前 [p.9]。
2. 强度削弱：将乘除等强运算替换为加减等弱运算 [p.10]。

3. 归纳变量消除：删除仅用于控制循环的基本归纳变量 [p.11]。

考点 6：数据流分析速查表

分析类型	方向	IN/OUT 公式	交汇运算	典型应用
到达定值	前向	$OUT = GEN \cup (IN - KILL)$	$\cup$ (并集)	常量传播、复写传播
活跃变量	后向	$IN = USE \cup (OUT - DEF)$	$\cup$ (并集)	寄存器分配
可用表达式	前向	$OUT = EGEN \cup (IN - EKILL)$	$\cap$ (交集)	全局 CSE

记忆诀窍：

- ”到达”和”可用”都是前向，但到达用并集（可能到达），可用用交集（每条路径都可用）。
- ”活跃”是后向，用并集（后向与前向”到达”对称）。
- 可用表达式与其他两个的区别在于用交集：因为”可用”是 must 属性（一定被计算），而”到达””活跃”是 may 属性（可能到达/活跃）。

考点 7：基本块划分算法

1. 找到所有入口语句（Leader）：首语句、转移目标语句、转移语句的下一条。
2. 每个入口语句到下一个入口之前（或程序结束）构成一个基本块。

考点 8：DAG 的作用

在基本块中构建 DAG（有向无环图）可以：

1. 自动检测公共子表达式（相同表达式 $\rightarrow$ 同一节点）。
2. 确定哪些值在基本块外被引用（输出节点 / 活跃变量）。
3. 确定哪些计算必须保留。

考点 9：常考判断题 / 简答题

1. Q: 代码外提是否总是安全？
A: 否。若循环可能不执行，外提的不变运算不能改变程序语义；外提的语句还需支配所有出口。
2. Q: 复写传播和常量传播的关系？
A: 常量传播是复写传播的特殊情况，源操作数为常量。

3. **Q:** 强度削弱和归纳变量消除的适用条件?

A: 需要循环中有归纳变量（每次迭代按固定量变化），且存在该归纳变量的线性派生变量。

4. **Q:** 可用表达式分析为何用交集而非并集?

A: 可用性是 must 属性：必须每条到达路径上表达式都被计算过才能保证值一定可用。并集会误将”可能可用”当作”一定可用”。